

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №3
по дисциплине «Математическая логика и теория
формальных языков»
«Реализация контекстно-свободной грамматики
подмножества естественного языка»

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Востров А. В. _____

«____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Описание грамматики	4
1.1 Частицы	4
1.2 Определения в именной группе	4
1.3 Формы предложений	5
1.3.1 Утвердительные предложения	5
1.3.2 Отрицательные предложения	5
1.3.3 Вопросительные предложения	5
2 Математическое описание	6
2.1 Грамматика и язык	6
2.2 Формальное определение	6
2.3 LL-грамматика и рекурсивный спуск	8
2.4 БНФ	8
3 Особенности реализации	11
3.1 Структуры данных	11
3.2 Функции	13
3.2.1 validate_sentence()	13
3.2.2 generate_sentence()	14
3.2.3 generate_subject()	14
3.2.4 generate_object()	15
3.2.5 generate_noun_phrase()	15
3.2.6 generate_time()	16
3.2.7 generate_adjective_list()	16
3.2.8 generate_verb_phrase()	16
3.2.9 generate_affirmative()	17
3.2.10 generate_negative()	17
3.2.11 Меню	18
4 Результаты работы	20
Заключение	21
Список литературы	22

Введение

В данной работе необходимо:

1. Выделить грамматику подмножества естественного языка.
2. Построить БНФ-нотацию выделенной грамматики.
3. Реализовать программу, которая:
 - (a) распознает принадлежность вводимого предложения к выбранной грамматике,
 - (b) генерирует предложение, соответствующее выбранной грамматике.

В качестве подмножества естественного языка было выбрано будущее время корейского языка.

1 Описание грамматики

Будущее время в корейском языке используется для выражения намерения говорящего или предположения о будущем действии. Оно отражает субъективное намерение или объективную догадку о действиях в будущем.

В корейском языке базовый порядок слов – Subject-Object-Verb (субъект-объект-глагол). Грамматика различных времен и конструкций строится путем присоединения окончаний к основе глагола.

1.1 Частицы

Для обозначения субъекта и объекта предложения существуют отдельные частицы.

- Субъект: 이/가
 - Если существительное заканчивается на согласную, используется частица 이.
 - Если существительное заканчивается на гласную, используется частица 가.
- Объект: 을/를
 - Если существительное заканчивается на согласную, используется частица 을.
 - Если существительное заканчивается на гласную, используется частица 를.

1.2 Определения в именной группе

Внутри именной группы определения (прилагательные и относительные конструкции) всегда предшествуют существительному.

Несколько прилагательных могут последовательно модифицировать одно существительное.

Помимо прилагательных, в корейском языке используются определительные конструкции, соответствующие придаточным предложениям в русском языке. Определительные придаточные предложения образуются без союзного слова. Глагол принимает специальную определительную форму, и вся конструкция ставится перед определяемым существительным.

Структура определительной конструкции: Subject + Object + Verb_Stem + relative_Ending.

Пример: 내가 책을 읽는 학생 — студент, который читает книгу.

1.3 Формы предложений

1.3.1 Утвердительные предложения

Будущее время в корейском обозначается окончанием глагола: -(을/ㄹ) + 거예요.

Выбор между 을 и ㄹ зависит от последней буквы основы:

- Последняя буква основы глагола – согласная: добавляем 을 거예요.
- Последняя буква основы – гласная или ㄷ: добавляем ㄹ 거예요.

Обстоятельства времени обычно ставятся в начале предложения.

Таким образом, утвердительные предложения образуются по схеме: Time + Subject + (이/가) + Object + (을/를) + Verb_Stem + (을/ㄹ) + 거예요 + ".", где Subject и Object являются именными группами, которые могут включать определения.

1.3.2 Отрицательные предложения

В корейском частица, означающая отрицание – 안, – ставится перед глаголом. Тогда предложение принимает форму: Time + Subject + (이/가) + Object + (을/를) + 안 + Verb_Stem + (을/ㄹ) + 거예요 + ".".

1.3.3 Вопросительные предложения

Порядок слов такой же, как в утвердительных предложениях, отличие лишь в интонации. Форма остается прежней: Time + Subject + (이/가) + Object + (을/를) + Verb_Stem + (을/ㄹ) + 거예요 + "?".

2 Математическое описание

2.1 Грамматика и язык

В данной работе строится *контекстно-свободная грамматика*, задающая язык предложений в будущем времени корейского языка.

2.2 Формальное определение

Порождающая грамматика Хомского определяется как:

$$G = \langle T, N, S, R \rangle, \text{ где}$$

T – конечное множество терминалов (слов языка),

N – конечное множество нетерминалов ($T \cap N = \emptyset$),

$S \in N$ – начальный нетерминал,

R – множество правил вывода вида $\alpha \rightarrow \beta$.

Грамматика является контекстно-свободной, если правила вывода имеют форму $A \rightarrow \gamma$, где A – один нетерминал, γ – последовательность терминалов и нетерминалов.

В данной работе:

- $T = \{ \text{“내일”, “다음”, “주”, “내년”, “나중에”, “곧”, “에”, “이”, “그”, “저”, “사람”, “학생”, “선생님”, “여자”, “남자”, “고양이”, “집”, “학교”, “책”, “편지”, “영화”, “나”, “너”, “그녀”, “우리”, “그들”, “가”, “을”, “를”, “예쁜”, “큰”, “작은”, “슬픈”, “행복한”, “화난”, “안”, “하”, “오”, “덥”, “보”, “만들”, “쓰”, “읽”, “을 거예요”, “르 거예요”, “는”, “은”, “ㄴ”, “을”, “르”, “.”, “?”} \}$
- $N = \{ \langle sentence \rangle, \langle affirmative \rangle, \langle negative \rangle, \langle question \rangle, \langle time_opt \rangle, \langle time \rangle, \langle time_particle_opt \rangle, \langle time_noun \rangle, \langle time_particle \rangle, \langle subject \rangle, \langle object \rangle, \langle noun_phrase \rangle, \langle base_np \rangle, \langle relative_clause \rangle, \langle object_opt \rangle, \langle determiner \rangle, \langle noun \rangle, \langle pronoun \rangle, \langle subject_particle \rangle, \langle object_particle \rangle, \langle adjective_list \rangle, \langle adjective \rangle, \langle verb_stem \rangle, \langle verb_base \rangle, \langle future_ending \rangle, \langle ending_cons \rangle, \langle ending_vowel_or_l \rangle, \langle relative_ending \rangle \}$
- $S = \langle sentence \rangle$

- В R входят следующие правила:

sentence $\rightarrow$ affirmative | negative | question
 affirmative $\rightarrow$ time_opt subject adjective_list object verb_stem
 future_ending “.”
 negative $\rightarrow$ time_opt subject adjective_list object “안” verb_stem
 future_ending “.”
 question $\rightarrow$ time_opt subject adjective_list object verb_stem
 future_ending “?”
 time_opt $\rightarrow$ time | ε
 time $\rightarrow$ time_noun time_particle_opt
 time_particle_opt $\rightarrow$ time_particle | ε
 time_noun $\rightarrow$ “내일” | “다음” | “주” | “내년” | “나중에” | “곧”
 time_particle $\rightarrow$ “에”
 subject $\rightarrow$ noun_phrase subject_particle
 object $\rightarrow$ noun_phrase object_particle
 noun_phrase $\rightarrow$ base_np | relative_clause base_np
 base_np $\rightarrow$ determiner noun | noun | pronoun
 relative_clause $\rightarrow$ subject object_opt verb_stem relative_ending
 object_opt $\rightarrow$ object | ε
 determiner $\rightarrow$ “이” | “그” | “저”
 noun $\rightarrow$ “사람” | “학생” | “선생님” | “여자” | “남자” | “고양이”
 | “집” | “학교” | “책” | “편지” | “영화”
 pronoun $\rightarrow$ “나” | “너” | “그” | “그녀” | “우리” | “그들”
 subject_particle $\rightarrow$ “이” | “가”
 object_particle $\rightarrow$ “을” | “를”
 adjective_list $\rightarrow$ adjective adjective_list | ε
 adjective $\rightarrow$ “예쁜” | “큰” | “작은” | “슬픈” | “행복한” | “화난”
 verb_stem $\rightarrow$ verb_base
 verb_base $\rightarrow$ “하” | “가” | “오” | “먹” | “보” | “만들” | “쓰” | “읽”
 future_ending $\rightarrow$ ending_cons | ending_vowel_or_l
 ending_cons $\rightarrow$ “을 거예요”
 ending_vowel_or_l $\rightarrow$ “ㄴ 거예요”
 relative_ending $\rightarrow$ “는” | “은” | “ㄴ” | “을” | “ㄴ”

2.3 LL-грамматика и рекурсивный спуск

LL-грамматика – контекстно-свободная грамматика, разбор которой выполняется слева направо с построением левого вывода.

LL(1)-грамматика – LL-грамматика, для которой выбор правила вывода может быть выполнен по одному символу предпросмотра входной цепочки.

Рекурсивный спуск является способом реализации LL-анализатора, при котором каждому нетерминалу грамматики соответствует отдельная процедура разбора. Разбор выполняется сверху вниз, начиная со стартового нетерминала.

В данной работе используется подход к синтаксическому анализу, близкий к рекурсивному спуску и LL-анализу, поскольку разбор выполняется сверху вниз на основе правил контекстно-свободной грамматики. Однако данная грамматика не является строго LL(1), так как содержит ϵ -продукции, рекурсивные конструкции и правила с одинаковыми префиксами, из-за чего выбор альтернативы невозможно выполнить по одному символу предпросмотра.

2.4 БНФ

```
<sentence>      ::= <affirmative> | <negative> | <question>

<affirmative>   ::= <time_opt> <subject> <adjective_list> <object>
                  <verb_stem> <future_ending> "."
<negative>      ::= <time_opt> <subject> <adjective_list> <object>
                  "안" <verb_stem> <future_ending> "."
<question>      ::= <time_opt> <subject> <adjective_list> <object>
                  <verb_stem> <future_ending> "?"

<time_opt>      ::= <time> |  $\epsilon$ 
<time>          ::= <time_noun> <time_particle_opt>
<time_particle_opt> ::= <time_particle> |  $\epsilon$ 

<time_noun>     ::= "내일" | "다음 주" | "내년" | "나중에" | "곧"
<time_particle> ::= "에"

<subject>       ::= <noun_phrase> <subject_particle>
<object>        ::= <noun_phrase> <object_particle>

<noun_phrase>   ::= <base_np> | <relative_clause> <base_np>
<base_np>       ::= <determiner> <noun> | <noun> | <pronoun>

<relative_clause> ::= <subject> <object_opt> <verb_stem>
```


<relative_ending>

<object_opt> ::= <object> | ε

<determiner> ::= "이" | "그" | "저"

<noun> ::= "사람" | "학생" | "선생님" | "여자" | "남자"
 | "고양이" | "집" | "학교" | "책" | "편지" | "영화"

<pronoun> ::= "나" | "너" | "그" | "그녀" | "우리" | "그들"

<subject_particle> ::= "이" | "가"

<object_particle> ::= "을" | "를"

<adjective_list> ::= <adjective> <adjective_list> | ε

<adjective> ::= "예쁜" | "큰" | "작은" | "슬픈"
 | "행복한" | "화난"

<verb_stem> ::= <verb_base>

<verb_base> ::= "하" | "가" | "오" | "먹" | "보" | "만들"
 | "쓰" | "읽"

<future_ending> ::= <ending_cons> | <ending_vowel_or_l>

<ending_cons> ::= "을 거예요"

<ending_vowel_or_l> ::= "ㄹ 거예요"

<relative_ending> ::= "는" | "은" | "ㄴ" | "을" | "ㄹ"

Примеры:

내일에 나가 영화를 볼 거예요. — Завтра я буду смотреть фильм.

곧 그가 큰 집을 만들 거예요. — Скоро он построит большой дом.

그녀가 편지를 안 쓸 거예요. — Она не будет писать письмо.

그가 책을 읽을 거예요? — Он будет читать книгу?

그녀가 편지를 쓰는 사람을 만날 거예요. — Она встретит человека, который пишет письмо.

내가 편지를 쓰는 사람이 읽는 책을 볼 거예요. — Я увижу книгу, которую читает человек, который пишет письмо.

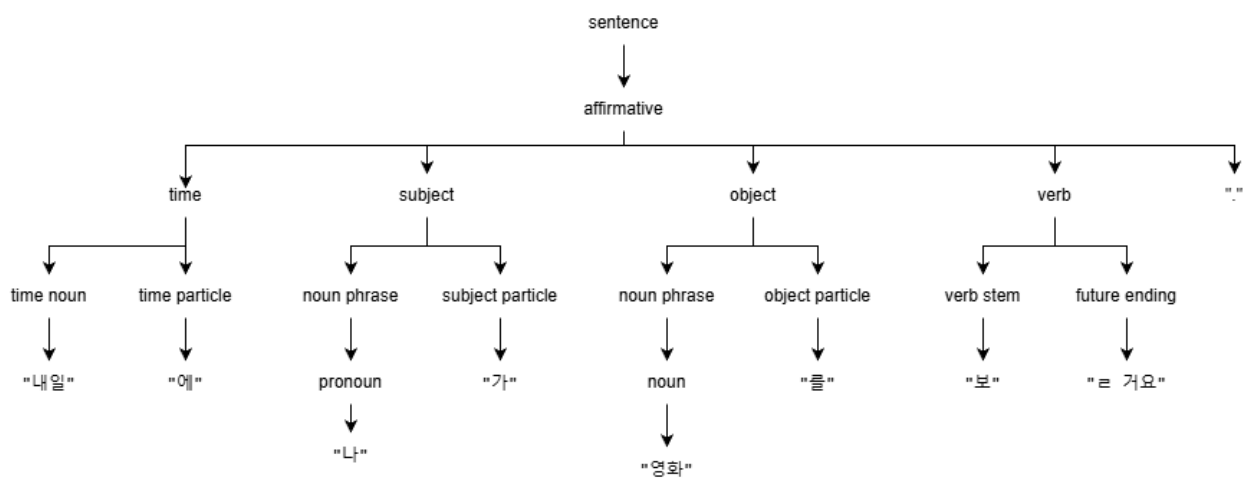


Рис. 1. Дерево разбора

3 Особенности реализации

3.1 Структуры данных

- Определения времени
time_nouns = ["내일", "다음", "주", "내년", "나중에", "곧"]
- Указательные определители
determiners = ["이", "그", "저"]
- Существительные
nouns = ["사람", "학생", "선생님", "여자", "남자", "고양이", "집", "학교", "책", "편지", "영화"]
- Местоимения
pronouns = ["나", "너", "그", "그녀", "우리", "그들"]
- Субъектные частицы
subject_particles = ["이", "가"]
- Объектные частицы
object_particles = ["을", "를"]
- Прилагательные
adjectives = ["예쁜", "큰", "작은", "슬픈", "행복한", "화난"]
- Основы глаголов
verb_bases = ["하", "가", "오", "먹", "보", "만들", "쓰", "읽"]
- Окончания будущего времени
future_endings = ["을 거예요", "ㄹ 거예요"]
- Окончания относительных конструкций
relative_endings = ["는", "은", "ㄴ", "을", "ㄹ"]

Правила грамматики хранятся в строке **GRAMMAR**. Она используется парсером для проверки принадлежности предложения языку.

Листинг 1. Строка GRAMMAR

```
1 GRAMMAR = r"""
2 start: sentence
3
4 sentence: affirmative
5           | negative
6           | question
7
8 affirmative: time_opt subject adjective_list object verb_phrase "."
9 negative: time_opt subject adjective_list object "안" verb_phrase "."
10 question: time_opt subject adjective_list object verb_phrase "?"
```

```

11
12 time_opt: time?
13 time: TIME_NOUN TIME_PARTICLE?
14
15 subject: noun_phrase SUBJECT_PARTICLE
16 object: noun_phrase OBJECT_PARTICLE
17
18 noun_phrase: base_np
19             | relative_clause base_np
20
21 base_np: determiner_noun
22         | NOUN
23         | PRONOUN
24
25 determiner_noun: DETERMINER NOUN
26
27 relative_clause: subject object_opt VERB_BASE RELATIVE_ENDING
28 object_opt: object?
29
30 adjective_list: adjective*
31
32 verb_phrase: VERB_BASE FUTURE_ENDING
33
34 adjective: ADJECTIVE
35
36 TIME_NOUN: "내일"
37           | "다음"
38           | "주"
39           | "내년"
40           | "나중에"
41           | "곧"
42
43 TIME_PARTICLE: "에"
44
45 DETERMINER: "이"
46            | "그"
47            | "저"
48
49 NOUN: "사람"
50      | "학생"
51      | "선생님"
52      | "여자"
53      | "남자"
54      | "고양이"
55      | "집"
56      | "학교"
57      | "책"
58      | "편지"
59      | "영화"
60
61 PRONOUN: "나"
62         | "너"

```

```

63         | "그"
64         | "그녀"
65         | "우리"
66         | "그들"
67
68 SUBJECT_PARTICLE: "이"
69                 | "가"
70
71 OBJECT_PARTICLE: "을"
72                 | "를"
73
74 ADJECTIVE: "예쁜"
75           | "큰"
76           | "작은"
77           | "슬픈"
78           | "행복한"
79           | "화난"
80
81 VERB_BASE: "하"
82           | "가"
83           | "오"
84           | "먹"
85           | "보"
86           | "만들"
87           | "쓰"
88           | "읽"
89
90 FUTURE_ENDING: "을 거예요"
91               | "ㄴ 거예요"
92
93 RELATIVE_ENDING: "는"
94                 | "은"
95                 | "ㄴ "
96                 | "을"
97                 | "ㄴ "
98
99 %import common.WS
100 %ignore WS
101 ""

```

3.2 Функции

3.2.1 validate_sentence()

Функция выполняет синтаксический разбор предложения при помощи парсера. Если предложение успешно разбирается по правилам грамматики, функция возвращает True. При возникновении ошибки парсинга возвращается False.

Вход: sentence – предложение для проверки (строка), БНФ.

Выход: True, если предложение соответствует грамматике, иначе False.

Листинг 2. Функция validate_sentence(sentence)

```
1 def validate_sentence(sentence):
2
3     try:
4         parser.parse(sentence)
5         return True
6
7     except Exception as e:
8         print(f"\nОшибка разбора:\n{e}")
9         return False
```

3.2.2 generate_sentence()

Случайным образом выбирает тип предложения: утвердительное, отрицательное или вопросительное. После этого вызывает соответствующую функцию генерации и добавляет знак препинания.

Вход: БНФ

Выход: сгенерированное предложение (строка).

Листинг 3. Функция generate_sentence()

```
1 def generate_sentence():
2
3     sentence_type = random.choice([
4         "affirmative",
5         "negative",
6         "question"
7     ])
8
9     if sentence_type == "affirmative":
10         return generate_affirmative() + "."
11
12     elif sentence_type == "negative":
13         return generate_negative() + "."
14
15     else:
16         return generate_question() + "?"
```

3.2.3 generate_subject()

Генерирует именную группу и присоединяет к ней субъектную частицу.

Вход: БНФ

Выход: подлежащее предложения (строка).

Листинг 4. Функция generate_subject()

```
1 def generate_subject():
2
```

```

3     return (
4         generate_noun_phrase()
5         + random.choice(subject_particles)
6     )

```

3.2.4 generate_object()

Генерирует именную группу и присоединяет к ней объектную частицу.

Вход: БНФ

Выход: дополнение предложения (строка).

Листинг 5. Функция generate_object()

```

1 def generate_object():
2
3     return (
4         generate_noun_phrase()
5         + random.choice(object_particles)
6     )

```

3.2.5 generate_noun_phrase()

Случайно выбирает один из трёх вариантов: существительное, местоимение, конструкцию «определитель + существительное». После выбора формирует соответствующую строку.

Вход: БНФ

Выход: именная группа (строка).

Листинг 6. Функция generate_noun_phrase()

```

1 def generate_noun_phrase():
2
3     choice = random.choice([
4         "noun",
5         "determiner_noun",
6         "pronoun"
7     ])
8
9     if choice == "noun":
10         return random.choice(nouns)
11
12     elif choice == "pronoun":
13         return random.choice(pronouns)
14
15     else:
16         return (
17             random.choice(determiners)
18             + random.choice(nouns)
19         )

```

3.2.6 generate_time()

Выбирает случайное временное слово. С вероятностью 50% добавляет временную частицу «||».

Вход: БНФ

Выход: обстоятельство времени (строка).

Листинг 7. Функция generate_time()

```
1 def generate_time():
2
3     result = random.choice(time_nouns)
4
5     if random.random() < 0.5:
6         result += "||"
7
8     return result
```

3.2.7 generate_adjective_list()

Случайным образом выбирает несколько прилагательных и объединяет их в строку через пробел.

Вход: БНФ

Выход: последовательность прилагательных (строка).

Листинг 8. Функция generate_adjective_list()

```
1 def generate_adjective_list():
2
3     result = []
4
5     count = random.randint(0, 2)
6
7     for _ in range(count):
8         result.append(random.choice(adjectives))
9
10    return " ".join(result)
```

3.2.8 generate_verb_phrase()

Случайно выбирает основу глагола и объединяет с окончанием будущего времени.

Вход: БНФ

Выход: глагольная группа (строка).

Листинг 9. Функция generate_verb_phrase()

```
1 def generate_verb_phrase():
2
3     return (
```



```

4         random.choice(verb_bases)
5         + random.choice(future_endings)
6     )

```

3.2.9 generate_affirmative()

Формирует предложение по схеме <time_opt> + <subject> + <adjective_list> + <object> + <verb_phrase>.

Вход: БНФ, результаты работы функций generate_time, generate_subject, generate_adjective_list, generate_object, generate_verb_phrase

Выход: утвердительное или вопросительное предложение без знака препинания (строка).

Листинг 10. Функция generate_affirmative()

```

1 def generate_affirmative():
2
3     parts = []
4
5     if random.random() < 0.5:
6         parts.append(generate_time())
7
8     parts.append(generate_subject())
9
10    adjectives_part = generate_adjective_list()
11
12    if adjectives_part:
13        parts.append(adjectives_part)
14
15    parts.append(generate_object())
16
17    parts.append(generate_verb_phrase())
18
19    return " ".join(parts)

```

3.2.10 generate_negative()

Формирует отрицательное предложение. После дополнения добавляется отрицательная частица **안**.

Вход: БНФ, результаты работы функций generate_time, generate_subject, generate_adjective_list, generate_object, generate_verb_phrase

Выход: отрицательное предложение без точки (строка).

Листинг 11. Функция generate_negative()

```

1 def generate_negative():
2
3     parts = []
4

```

```

5     if random.random() < 0.5:
6         parts.append(generate_time())
7
8     parts.append(generate_subject())
9
10    adjectives_part = generate_adjective_list()
11
12    if adjectives_part:
13        parts.append(adjectives_part)
14
15    parts.append(generate_object())
16
17    parts.append("안")
18
19    parts.append(generate_verb_phrase())
20
21    return " ".join(parts)

```

3.2.11 Меню

Меню реализовано через цикл `while`. Пользователю предлагается выбор: сгенерировать предложение, соответствующее грамматике, или проверить предложение на соответствие грамматике. При некорректном вводе выводится сообщение об ошибке.

Листинг 12. Меню

```

1  while True:
2
3      print("\n1 - Сгенерировать предложение")
4      print("2 - Проверить предложение")
5      print("0 - Выход")
6
7      choice = input("Выберите действие: ")
8
9      if choice == "1":
10
11         sentence = generate_sentence()
12
13         print("\nСгенерированное предложение:")
14         print(sentence)
15
16     elif choice == "2":
17
18         sentence = input("Введите предложение: ")
19
20         result = validate_sentence(sentence)
21
22         print(
23             "Предложение соответствует грамматике."
24             if result
25             else

```

```
26         "Предложение НЕ соответствует грамматике."  
27     )  
28  
29     elif choice == "0":  
30         break  
31  
32     else:  
33         print("Неверный выбор.")
```

4 Результаты работы

Ниже представлены результаты работы программы.

Генерация предложений:

Сгенерированное предложение:

선생님이 작은 화난 너를 하ㄴ 거예요.

Сгенерированное предложение:

다음에 우리가 큰 그여자를 안 하ㄴ 거예요.

Сгенерированное предложение:

내일 학교가 작은 화난 그들을 읽을 거예요?

Проверка предложений на соответствие грамматике:

Введите предложение: 다음에 우리가 큰 그여자를 안 하ㄴ 거예요.

Предложение соответствует грамматике.

Введите предложение: 선생님이 작은 화난 너를 하ㄴ 거예요.

Предложение соответствует грамматике.

Введите предложение: hjbsfjvbsk

Предложение НЕ соответствует грамматике.

Введите предложение: 선생님이 작은

Предложение НЕ соответствует грамматике.

Заключение

В ходе данной работы была построена контекстно-свободная грамматика будущего времени корейского языка, на основе этой грамматики была построена БНФ-нотация, а также реализована программа, которая распознает принадлежность вводимого предложения к выбранной грамматике, и генерирует предложение, соответствующее выбранной грамматике.

Построенная грамматика принадлежит к 2 типу по иерархии Хомского. Анализатор не является строго LL(1)-анализатором, однако использует подход близкий к рекурсивному спуску и LL-анализу.

Достоинства:

1. Программа поддерживает генерацию разных типов предложений, а также учитывает возможность отсутствия некоторых частей.
2. Возможность дальнейшего расширения без существенного изменения архитектуры программы.

Недостатки:

1. Ограниченный словарный запас.
2. Из-за случайной генерации предложения могут получаться правильными грамматически, но странными по смыслу.
3. Грамматика описывает только ограниченный набор конструкций будущего времени.

Масштабирование:

1. Добавление ввода из файла.
2. Расширение словаря.
3. Добавление новых типов предложений и грамматических конструкций.

Список литературы

1. Секция «Телематика» – <https://tema.spbstu.ru/compiler/> (дата обращения – 04.05.2026)
2. Кён Хва Сон, Стефани Бэйтс Talk to me in Korean 3 — 9-е изд. — Сеул: Longtail Books, 2020 — 200 с.